

# PrimeNet Project Documentation

## 1. Project Overview

PrimeNet is a Flask-based internal web platform for telecom network operations. It combines:

- Network performance analytics (multi-vendor, multi-technology KPIs)
- Network metadata visualization and map analysis
- Configuration tooling (XML parsing, Excel generation, NE comparison)
- Sync and ingestion pipelines from SFTP sources into local SQLite databases
- Administrative and user workflows (auth, roles, profile, task scheduler, reports)

The project is structured as a modular Flask monolith where each feature area lives in a blueprint package under ``modules/``.

---

## 2. High-Level Architecture

### *2.1 Runtime model*

- Framework: Flask
- Storage backend: SQLite-only (no active PostgreSQL path)
- Background jobs: APScheduler
- Authentication: cookie session token, server-side session table
- Data source integration: SFTP pulls + local raw loaders

### *2.2 Main startup sequence*

``app.py`` is the runtime entrypoint and performs:

1. Flask app creation and core config (``MAX_CONTENT_LENGTH``, ``SECRET_KEY``)
2. Blueprint registration for all feature modules
3. Data DB migrations (``modules.sync.db_migration.run_migrations()``)
4. App DB initialization (``database_enhanced.init_db()``, ``create_admin_user()``)
5. Scheduler startup (``modules.sync.scheduler.start_scheduler()``) unless disabled
6. Password-rotation enforcement with ``@app.before_request``

### *2.3 Blueprint composition*

The app imports and registers:

- ``routes.auth_routes` (`auth_bp`)`
- ``modules.xml_parser``
- ``modules.excel_generator``
- ``modules.ne_comparison``
- ``modules.parameter_dictionary``
- ``modules.network_map``
- ``modules.admin_panel``
- ``modules.sync``
- ``modules.performance``
- ``modules.config_history``
- ``modules.network_management``
- ``modules.reports``
- ``modules.conflict_map``
- ``modules.user_profile``
- ``modules.femto_pm``
- ``modules.task_scheduler``
- ``modules.drive_test_viewer``

---

### 3. Repository Structure

Key top-level Python and package areas:

- ``app.py``: Flask app bootstrap and registration
- ``database_enhanced.py``: app/user/task/profile/report DB schema + auth/session logic
- ``db/``: database runtime adapter and connection helpers
- ``sync_config.py``: environment/runtime config, DB paths, SFTP settings, ingestion knobs
- ``routes/auth_routes.py``: login/logout/dashboard/auth APIs
- ``modules/``: feature blueprints and supporting logic
- ``scripts/``: operational pull/load/migration/debug tooling
- ``ncm_core.py``: shared conversion/comparison utilities used by XML/Excel/NE modules

---

## 4. Authentication, Session, and Access Control

### 4.1 Auth flow

Defined mainly in ``routes/auth_routes.py`` + ``database_enhanced.py``:

- Login API: ``POST /api/login``
- Validates credentials (``authenticate_user``)
- Creates a DB-backed session (``create_session``)
- Sets ``session_token`` cookie (``httponly=True``)
- Logout API: ``POST /api/logout``
- Deletes session row (``delete_session``)
- Clears cookie
- Registration is intentionally disabled (``/register`` redirects, ``/api/register`` returns 403)

## 4.2 Session model

``database_enhanced.py`` tables:

- ``users``
- ``sessions``
- ``activity_log``

Session lookup uses ``get_user_by_session(session_token)`` and checks:

- Token exists
- Session not expired
- User active

## 4.3 Password policy

Global gate in ``app.py`` (``enforce_password_rotation``) and helper in ``database_enhanced.py``:

- ``is_password_change_required(user, max_days=60)``
- Enforced for web and API requests except explicit allowlist routes
- API requests get ``403`` with ``password_change_required=true``

## 4.4 Roles and authorization

Role checks are implemented per module via decorators/wrappers:

- ``admin`` required for sync APIs
- admin/operator roles in admin panel and scheduling flows
- task/profile/report operations scoped by user role and ownership

---

## 5. Database Layer and Storage Design

### 5.1 Runtime adapter (`db/runtime.py`)

Important points:

- `is\_postgresql() -> False` (hardcoded)
- SQLite connection helpers:
- `connect\_app()`
- `connect\_metadata()`
- `connect\_nokia\_pm()`
- `connect\_huawei\_pm()`
- Uses WAL + busy timeout + retry on `database is locked`
- `performance\_meta\_pm\_conn(vendor)` attaches PM DB(s) into metadata connection

This is optimized for read-heavy dashboard/API usage while background writes happen.

### 5.2 DB path authority (`sync\_config.py`)

All DB files are stored under `databases/`:

- `databases/admin/ncm\_users.db`
- `databases/metadata/metadata.db`
- `databases/cells/nokia\_pm\_cells.db`
- `databases/cells/huawei\_pm\_cells.db`
- Daily and groups DB variants
- Neighbor DBs under vendor-specific subdirectories

`\_migrate\_legacy\_db\_names()` handles compatibility from older file locations/names.

### 5.3 App schema (`database\_enhanced.py`)

Core tables created in `init\_db()` include:

- User/session/audit:
- `users`, `sessions`, `activity\_log`
- Tasking:
- `tasks`, `task\_updates`

- Profiles/preferences:
- ``filter_profiles`, `user_preferences``
- Feature storage:
- ``config_versions`, `report_archive``
- ``config_scheduler_tasks`, `config_scheduler_task_files`, `config_scheduler_result_files``

---

## 6. Sync and Ingestion Subsystem

The sync subsystem is spread across ``modules/sync/`` plus scripts.

### 6.1 Components

- ``modules/sync/routes.py``: admin APIs for status/history/triggers/diagnostics
- ``modules/sync/scheduler.py``: APScheduler jobs and trigger helpers
- ``modules/sync/pm_processor.py``: PM file parsing and DB ingestion
- ``modules/sync/metadata_processor.py``: metadata ingest and normalization
- ``modules/sync/group_processor.py``: group ingest (currently reset/disabled pathways)
- ``modules/sync/sftp_client.py``: SFTP utility class
- ``modules/sync/db_migration.py``: schema migration/bootstrap for data DBs

### 6.2 Scheduler behavior

``start_scheduler()``:

- Always runs migration bootstrap first
- Starts scheduler daemon
- Legacy periodic pull/load jobs are opt-in via ``NCM_ENABLE_LEGACY_PERFORMANCE_SCHEDULER``
- Optional remote pull watcher controlled by ``NCM_DISABLE_PULL_WATCHER``

Important implementation nuance:

- Some legacy direct pull methods (``pull_nokia_pm`, `pull_huawei_pm`, `pull_metadata`, group pulls`) currently return immediately in reset mode. Active sync behavior is primarily through master raw+loader script flows and explicit trigger paths.

### 6.3 Sync API surface

From ``modules/sync/routes.py``:

- Status and monitoring:
- `GET /api/sync/status`
- `GET /api/sync/progress`
- `GET /api/sync/history`
- Manual triggers:
- `POST /api/sync/trigger/pm`
- `POST /api/sync/trigger/nokia\_pm`
- `POST /api/sync/trigger/huawei\_pm`
- `POST /api/sync/trigger/metadata`
- Hourly/daily category triggers for cells/groups
- Utility/admin:
- `POST /api/sync/import\_pm\_path`
- `GET /api/sync/test`
- `GET /api/sync/inspect\_local`
- `GET /api/sync/latest\_downloads`

All are admin-restricted by `admin\_required`.

## 6.4 Sync logging

`sync\_log` table is used by both scheduler and APIs for:

- Trigger audits
- Pull/load result statuses
- Rows affected
- Message/error context

---

## 7. Module-by-Module Feature Documentation

### 7.1 `modules/admin\_panel`

Purpose:

- Admin UI and APIs for user management and PM recency checks

Endpoints:

- `GET /admin-panel`

- `GET /api/admin/users`
- `PUT /api/admin/users/<user\_id>/role`
- `PUT /api/admin/users/<user\_id>/status`
- `GET /api/admin/pm-latest-timestamps`

Key internals:

- Uses `database\_enhanced` user APIs
- Applies role checks and role-label mappings
- Exposes PM freshness diagnostics for operational monitoring

## 7.2 `modules/config\_history`

Purpose:

- Store and compare configuration XML versions by NE

Endpoints:

- `GET /config-history`
- `POST /api/config-history/upload`
- `GET /api/config-history/list`
- `GET /api/config-history/<ne\_name>/versions`
- `POST /api/config-history/diff`
- `GET /api/config-history/version/<id>/download`
- `DELETE /api/config-history/version/<id>`

Storage:

- `config\_versions` table in app DB

## 7.3 `modules/conflict\_map`

Purpose:

- Detect and visualize potential PCI conflicts with geospatial risk logic

Endpoints:

- `GET /conflict-map`
- `GET /api/conflict-map/data`
- `POST /api/conflict-map/refresh`

- `GET /api/conflict-map/export-kml`

Implementation notes:

- Logic in `modules/conflict\_map/logic.py`
- Uses strictness profiles (strict/standard/moderate/relaxed)
- Risk scoring based on distance/bearing/sector alignment heuristics

## 7.4 `modules/drive\_test\_viewer`

Purpose:

- Drive-test file ingestion and map-oriented parsing

Endpoints:

- `GET /drive-test-viewer`
- `POST /api/drive-test-viewer/upload`

Implementation notes:

- Parses field artifacts (GPX/NMFS related flows)
- Converts uploaded data to frontend-consumable trace payloads

## 7.5 `modules/excel\_generator`

Purpose:

- Convert Excel inputs into XML outputs

Endpoints:

- `GET /excel-generator`
- `POST /api/excel-generator/upload`
- `POST /api/excel-generator/convert`
- `GET /api/excel-generator/download/<filename>`

Core dependency:

- `ncm\_core.ExcelToXMLConverter`

## 7.6 `modules/femto\_pm`



Purpose:

- Femto PM analytics and trend visualization

Endpoints:

- `GET /femto-pm`
- `GET /api/femto-pm/devices`
- `GET /api/femto-pm/catalog`
- `GET /api/femto-pm/kpi-columns`
- `GET /api/femto-pm/trend`

Implementation notes:

- Reads femto PM SQLite tables
- Supports KPI expression/formula evaluation

## 7.7 `modules/ne\_comparison`

Purpose:

- Compare NE XML files and produce report outputs

Endpoints:

- `GET /ne-comparison`
- `POST /api/ne-comparison/compare`
- `POST /api/ne-comparison/download-report`

Core dependency:

- `ncm\_core.XMLComparator`

## 7.8 `modules/network\_management`

Purpose:

- Site/cell browsing and PCI conflict checks over metadata

Endpoints:

- `GET /network-management`
- `GET /api/network-management/pci-conflicts`

- `GET /api/network-management/sites`
- `GET /api/network-management/site/<site\_id>/cells`
- `GET /api/network-management/summary`

## 7.9 `modules/network\_map`

Purpose:

- Main geospatial network map and neighbor analysis APIs

Primary endpoints:

- Pages:
- `GET /network-map`
- `GET /neighbor-analysis`
- Data:
- `GET /api/map/sites`
- `GET /api/map/tech-filter-options`
- `GET /api/map/site/<site\_id>`
- `GET /api/map/cells/wedge-data`
- `GET /api/map/cell/<cell\_id>/kpi`
- `GET /api/map/cell/kpi`
- `GET /api/map/stats`
- `GET /api/map/search/cell-code`
- Export:
- `GET /api/map/export/cell-code`
- `GET /api/map/export/sites`
- `GET /api/map/export/kml`
- Neighbor analysis:
- `GET /api/network-map/neighbors/lines`
- `GET /api/network-map/neighbors/cell-summary`
- Maintenance:
- `POST /api/map/refresh`

Implementation notes:

- Uses metadata unions and neighbor KPI datasets
- Generates map-friendly payloads and KML export structures

## 7.10 `modules/parameter\_dictionary`

Purpose:

- Explore/search managed object parameters and descriptions

Endpoints:

- `GET /parameter-dictionary`
- `GET /api/parameter-dictionary/list`
- `POST /api/parameter-dictionary/search`

## 7.11 `modules/performance`

Purpose:

- Main KPI analytics engine and report/filter APIs

Primary endpoints:

- `GET /performance`
- `GET /api/performance/kpi\_columns`
- `GET /api/performance/kpi\_headers\_map`
- `GET /api/performance/kpi\_mapping`
- `GET /api/performance/groups`
- `GET /api/performance/groups/<group\_ref>/cell\_keys`
- `GET /api/performance/group/trend`
- `GET /api/performance/reports`
- `POST /api/performance/reports`
- `DELETE /api/performance/reports/<id>`
- `GET /api/performance/filters`
- `GET /api/performance/cells`
- `GET /api/performance/cell/<cell\_id>/trend`
- `GET /api/performance/cell/trend`
- `GET /api/performance/pm-table`
- Additional sync trigger helpers under `api/sync/trigger/nokia|huawei`

Implementation notes:

- Largest route module by code size
- Heavy dynamic table/column introspection

- Handles mixed-vendor KPI datasets and adaptable schema evolution

## 7.12 `modules/reports`

Purpose:

- Generate, download, and archive operational reports

Endpoints:

- `GET /reports`
- `POST /api/reports/generate`
- `GET /api/reports/download/<report\_id>`
- `GET /api/reports/archive`
- `DELETE /api/reports/archive/<report\_id>`
- `GET /api/reports/types`

Supporting code:

- `modules/reports/metadata\_helpers.py`

Implementation notes:

- Includes metadata union helpers
- Uses elevation caching/lookup logic in report workflows

## 7.13 `modules/task\_scheduler`

Purpose:

- Configuration task lifecycle with file attachments and result delivery

Endpoints:

- `GET /config-task-scheduler`
- `GET /api/config-task-scheduler/tasks`
- `POST /api/config-task-scheduler/tasks`
- `DELETE /api/config-task-scheduler/tasks/<task\_id>`
- `POST /api/config-task-scheduler/tasks/<task\_id>/complete`
- `POST /api/config-task-scheduler/tasks/<task\_id>/status`
- `GET /api/config-task-scheduler/tasks/<task\_id>/file/<file\_id>/download`
- `GET /api/config-task-scheduler/tasks/<task\_id>/result/<file\_id>/download`

Storage:

- `config\_scheduler\_`\* tables in app DB

## 7.14 `modules/user\_profile`

Purpose:

- User profile, password, preferences, activity, and photo request workflows

Endpoints:

- `GET /profile`
- `GET /api/profile`
- `POST /api/profile/update`
- `POST /api/profile/photo-request`
- `GET /api/profile/photo-requests`
- `POST /api/profile/photo-requests/<req\_id>/review`
- `POST /api/profile/change-password`
- `GET /api/profile/preferences`
- `POST /api/profile/preferences`
- `GET /api/profile/activity`

## 7.15 `modules/xml\_parser`

Purpose:

- Convert XML inputs to Excel outputs

Endpoints:

- `GET /xml-parser`
- `POST /api/xml-parser/upload`
- `POST /api/xml-parser/convert`
- `GET /api/xml-parser/download/<filename>`

Core dependency:

- `ncm\_core.XMLToExcelConverter`

---

## 8. Configuration (`sync\_config.py`) Deep Dive

`sync\_config.py` centralizes:

- Environment loading from `.env`
- DB file path constants and directory creation
- PM/group/metadata scheduler timing controls
- Runtime ingestion options:
- `PM\_SYNC\_MODE`
- `PM\_RETENTION\_DAYS`
- `PM\_INSERT\_BATCH\_SIZE`
- `RAW\_LOADER\_INCREMENTAL`
- `RAW\_LOADER\_TIME\_FILTER`
- `RAW\_PULL\_INTERVAL\_HOURS`
- `DAILY\_PULL\_HOUR`
- `PULL\_WATCHER\_POLL\_INTERVAL\_SEC`
- `SYNC\_DOWNLOAD\_KEEP\_FILES`
- SFTP server definitions for:
- Nokia PM (+ daily + groups + neighbors)
- Huawei PM (+ daily + groups + neighbors)
- Metadata snapshots
- Legacy/static KPI header maps for import utilities

---

## 9. Scripts Directory Documentation

`scripts/` contains operational tooling grouped into categories.

### 9.1 Pull orchestrators

- `pull\_all\_raw.py`
- `pull\_all\_raw\_daily.py`
- `pull\_and\_load\_daily.py`

### 9.2 Vendor/source pulls

- PM and metadata:
- `pull\_nokia\_raw.py`, `pull\_huawei\_raw.py`

- ``pull_nokia_raw_daily.py`, `pull_huawei_raw_daily.py``
- ``pull_metadata_raw.py`, `pull_femto_raw.py``
- Neighbor:
- ``pull_nokia_neighbor_raw.py`, `pull_huawei_neighbor_raw.py``
- ``pull_performance_project_neighbor.py``

## 9.3 Loaders and importers

- ``load_raw_csv_to_databases.py``
- ``load_raw_daily_to_databases.py``
- ``load_nokia_neighbor_raw_to_db.py``
- ``load_huawei_neighbor_wide_to_db.py``
- ``load_neighbor_reports.py``
- ``load_femto_pm_to_db.py``
- ``import_femto_catalogs.py``
- ``parse_femto_metadata.py``
- ``import_local_files.py`` (staged local import pipeline)

## 9.4 Schema/bootstrap/migration utilities

- ``init_database.py``
- ``add_new_tables.py``
- ``add_map_tables.py``
- ``build_neighbor_kpi_db.py``
- ``build_kpi_headers_db.py``
- ``drop_legacy_performance_storage.py``

## 9.5 Diagnostics/debug tools

- ``watch_remote_new_files_and_pull.py``
- ``audit_sqlite_databases.py``
- ``dump_db_headers.py``
- ``check_neighbor_dbs.py``
- ``drop_duplicate_kpis.py``
- ``_inspect_huawei_now.py``
- ``_reload_huawei_only.py``
- ``_probe_trend_flow.py``
- ``debug_huawei_pm_zip.py``

---

## 10. Data Flow Walkthroughs

### 10.1 User/API flow

1. Client requests login
2. Auth verifies user and creates session row
3. Cookie (`session\_token`) stored in browser
4. Protected endpoints resolve user from session
5. Role-specific checks allow/deny feature operations

### 10.2 Sync ingestion flow (legacy-enabled path)

1. Scheduler or admin trigger runs master pull scripts
2. Raw files land in local `sync\_downloads` / `raw` locations
3. Loader scripts parse CSV/XLSX/ZIP files
4. Data inserted/upserted into PM/group/metadata SQLite DBs
5. `sync\_log` receives detailed status/rows/errors
6. APIs (`performance`, `network\_map`, `reports`) query updated DBs

### 10.3 Reporting flow

1. User requests report generation
2. Reports module reads metadata + KPI data via helper queries
3. Report artifact generated and archived in `report\_archive`
4. User downloads report via report ID endpoint

---

## 11. Operational Controls and Environment Variables

Common runtime flags:

- `NCM\_DISABLE\_SCHEDULER=1` to disable APScheduler startup
- `NCM\_ENABLE\_LEGACY\_PERFORMANCE\_SCHEDULER=1` to enable legacy periodic pull/load jobs
- `NCM\_DISABLE\_PULL\_WATCHER=1` to disable remote signature watcher job
- `PM\_SYNC\_MODE` (`full` or `incremental`)
- `PM\_RETENTION\_DAYS`, `FEMTO\_RETENTION\_DAYS`



- ``RAW_PULL_INTERVAL_HOURS`, `DAILY_PULL_HOUR``
- ``RAW_LOADER_INCREMENTAL`, `RAW_LOADER_TIME_FILTER``

---

## 12. Error Handling and Resilience Patterns

Observed patterns in code:

- SQLite lock mitigation:
- WAL mode + busy timeout
- retry loops on lock errors
- API-level defensive guards:
- role/session checks
- broad try/except wrappers around external I/O
- Sync auditability:
- central ``sync_log`` writes for command and job outcomes
- Startup tolerance:
- migration/init/scheduler failures are caught and logged as warnings

---

## 13. Security and Hardening Notes

Current implementation characteristics that should be documented for production hardening:

- ``SECRET_KEY`` in ``app.py`` is currently static placeholder text
- SFTP credentials and hosts are present in ``sync_config.py`` (sensitive configuration in code)
- Password hashing uses salted SHA-256; consider stronger adaptive hash (Argon2/bcrypt/scrypt)
- Session cookie is ``httponly``, but additional flags (``secure``, ``samesite``) should be explicitly set per deployment profile
- Default admin creation is automatic on first startup with static credentials if DB is empty

Recommended approach:

- Move secrets and credentials to environment variables or secret manager
- Rotate any credentials that have ever been committed
- Add deployment-specific security config profile

---

## 14. Known Architectural Characteristics

- Monolithic Flask app with many route-heavy modules
- SQLite-first architecture optimized for local/internal deployment
- Heavy analytics logic embedded in blueprint route files
- Sync subsystem has both current and legacy execution paths (feature-flag gated)
- Data model supports both operational tooling and analytics/reporting workloads

---

## 15. Suggested Next Documentation Expansions

To make this fully enterprise-grade documentation set, add:

- Separate API reference (OpenAPI/Swagger) generated from route definitions
- Data dictionary for each major SQLite DB and table
- Sequence diagrams for:
  - login/session lifecycle
  - sync scheduler cycle
  - report generation flow
- Deployment playbook (dev/stage/prod profiles)
- Incident runbook for sync failures and DB lock contention
- Test strategy document by module and data pipeline stage

---

## 16. Quick Start (Code Reader's Guide)

If you are new to the codebase, read in this order:

1. ``app.py``
2. ``routes/auth_routes.py``
3. ``database_enhanced.py``
4. ``db/runtime.py``
5. ``sync_config.py``
6. ``modules/sync/routes.py`` and ``modules/sync/scheduler.py``
7. ``modules/performance/routes.py``
8. ``modules/network_map/routes.py``
9. Remaining module ``routes.py`` files as needed by domain

This order gives a complete mental model of runtime, security, data, ingestion, then feature surfaces.